

Interprocedural Analysis

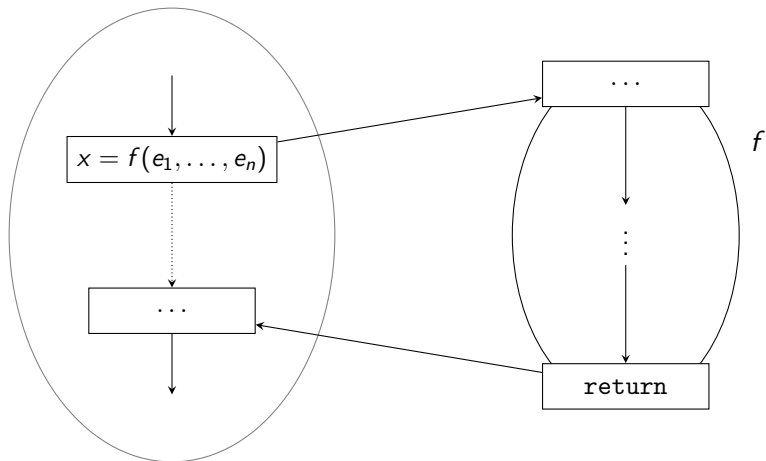
CSE552 Program Analysis — Lecture 10

Jaemin Hong

Interprocedural Analysis

- ▶ Intraprocedural analysis: analyzes the body of each individual function in isolation
- ▶ Interprocedural analysis: analyzes the whole program containing multiple functions and function calls

CFGs for Interprocedural Analysis



Transfer Functions — Call Node

$x = f(e_1, \dots, e_n)$:

$$t_v(\sigma) = \left\{ \begin{array}{l} ([x_1 \mapsto \text{eval}(e_1, \sigma), \dots, x_n \mapsto \text{eval}(e_n, \sigma)], \text{entry}(f)), \\ (\sigma[x \mapsto \sigma_{\text{return}}(\text{RET})], \text{after}(v)) \end{array} \right\}$$

where

- ▶ $x_1, \dots, x_n$ are the parameters of f
- ▶ $\text{entry}(f)$ is the entry node of f
- ▶ σ_{return} is the state at the return node of f
- ▶ $\text{after}(v)$ is the after-call node

Transfer Functions — Return Node

return:

$$t_v(\sigma) = \{(\sigma_{v_1}[x_1 \mapsto \sigma(\text{RET})], \text{after}(v_1)), \dots\}$$

where

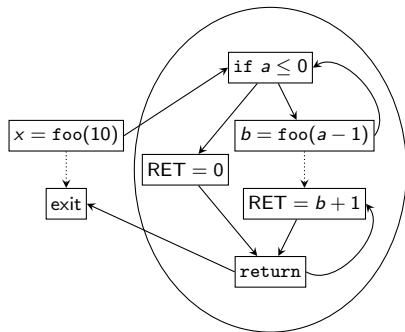
- ▶ v_i is a call node
- ▶ σ_{v_i} is the state at v_i
- ▶ x_i is the variable assigned the return value at v_i
- ▶ $\text{after}(v_i)$ is the after-call node for v_i

Example 1

```
fn foo(a) {  
    // a: [1, 1]  
    RET = a + a;  
    // RET: [2, 2]  
    return;  
}  
  
x = 1;  
// x: [1, 1]  
y = foo(x);  
// x: [1, 1], y: [2, 2]  
z = x + y;  
// z: [3, 3]
```

Example 2 — Code and CFG

```
fn foo(a) {  
  if a <= 0 {  
    RET = 0;  
  } else {  
    b = foo(a - 1);  
    RET = b + 1;  
  }  
  return;  
}  
x = foo(10);
```



Example 2 — Analysis Iteration (1)

```
fn foo(a) {  
  // a: [10, 10]  
  if a <= 0 {  
  
    RET = 0;  
  
  } else {  
    // a: [10, 10]  
    b = foo(a - 1);  
    RET = b + 1;  
  }  
  
  return;  
}  
  
x = foo(10);
```

```
fn foo(a) {  
  // a: [-inf, 10]  
  if a <= 0 {  
    // a: [-inf, 0]  
    RET = 0;  
    // RET: [0, 0]  
  } else {  
    // a: [1, 10]  
    b = foo(a - 1);  
    RET = b + 1;  
  }  
  // RET: [0, 0]  
  return;  
}  
  
x = foo(10);
```

Example 2 — Analysis Iteration (2)

```
fn foo(a) {  
  // a: [-inf, 10]  
  if a <= 0 {  
    // a: [-inf, 0]  
    RET = 0;  
    // RET: [0, 0]  
  } else {  
    // a: [1, 10]  
    b = foo(a - 1);  
    // b: [0, 0]  
    RET = b + 1;  
    // RET: [1, 1]  
  }  
  // RET: [0, 1]  
  return;  
}  
  
x = foo(10);
```

```
fn foo(a) {  
  // a: [-inf, 10]  
  if a <= 0 {  
    // a: [-inf, 0]  
    RET = 0;  
    // RET: [0, 0]  
  } else {  
    // a: [1, 10]  
    b = foo(a - 1);  
    // b: [0, inf]  
    RET = b + 1;  
    // RET: [1, inf]  
  }  
  // RET: [0, inf]  
  return;  
}  
  
x = foo(10);  
// x: [0, inf]
```

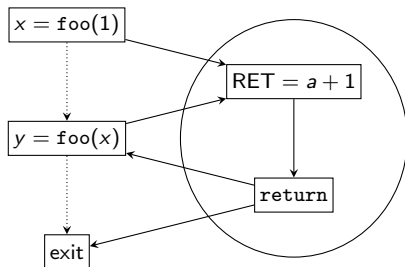
Example 3

```
fn foo(a) {  
  // a: [1, 2]  
  RET = a + a;  
  // RET: [2, 4]  
  return;  
}  
  
if input() {  
  x = 1;  
  // x: [1, 1]  
  y = foo(x);  
  // y: [2, 4]  
} else {  
  x = 2;  
  // x: [2, 2]  
  z = foo(x);  
  // z: [2, 4]  
}
```

Example 4 — Code and CFG

```
fn foo(a) {  
    RET = a + 1;  
    return;  
}
```

```
x = foo(1);  
y = foo(x);
```



Example 4 — Analysis Iterations

```
fn foo(a) {  
  // a: [1, 1]  
  RET = a + 1;  
  // RET: [2, 2]  
  return;  
}  
  
x = foo(1);  
// x: [2, 2]  
y = foo(x);  
// y: [2, 2]
```

```
fn foo(a) {  
  // a: [1, inf]  
  RET = a + 1;  
  // RET: [2, inf]  
  return;  
}  
  
x = foo(1);  
// x: [2, inf]  
y = foo(x);  
// y: [2, inf]
```

Context Sensitivity

- ▶ Context-insensitive analysis
 - ▶ Does not distinguish between different calls to the same function
 - ▶ Heavily suffers from interprocedurally invalid paths
 - ▶ e.g., dataflow from one call node propagates to all after-call nodes
- ▶ Context-sensitive analysis
 - ▶ Distinguishes different calls
 - ▶ $(\text{Context} \rightarrow (\text{State} \cup \{\text{unreachable}\}))^n$
 - ▶ If $\text{Context} = \{\cdot\}$, it is context-insensitive

Context-Sensitive Transfer Functions

$$t_v : \text{State} \times \text{Context} \rightarrow \mathcal{P}(\text{State} \times \text{Node} \times \text{Context})$$

$x = e$:

$$t_v(\sigma, c) = \begin{cases} \{(\text{unreachable}, \text{succ}(v), c)\} & \text{if } \sigma = \text{unreachable} \\ \{(\sigma[x \mapsto \text{eval}(\sigma, e)], \text{succ}(v), c)\} & \text{if } \sigma \neq \text{unreachable} \end{cases}$$

Fixed Point Algorithm

PropagationWithWideningAndContexts($t_1, \dots, t_n, \sigma_{start}$):

$((x_{11}, \dots, x_{1m}), \dots) := ((\sigma_{start}, \perp, \dots, \perp), \dots)$

$W := \{(v_1, c_1), \dots, (v_1, c_m), \dots\}$

while $W \neq \emptyset$:

$(v_i, c_j) := W.removeOne()$

$Y := t_{v_i}(x_{ij}, c_j)$

for $(y, v_k, c_l) \in Y$:

$z := x_{kl} \nabla y$

if $x_{kl} \neq z$:

$x_{kl} := z$

$W.add((v_k, c_l))$

return x

Call String Approach — Overview

- ▶ k -call-site sensitivity
- ▶ $\text{Call} = \{v \in \text{Node} \mid v \text{ is a call node}\}$
- ▶ $\text{Context} = \text{Call}^{\leq k}$ where $k \in \mathbb{Z}^+$
 - ▶ $(v_1, v_2, \dots, v_m)$ identifies the topmost m call sites
- ▶ The abstract state at $(v, (v_1, v_2, \dots, v_m))$ approximates the runtime state at v , where
 - ▶ The function containing v is called from v_1
 - ▶ The function containing v_1 is called from v_2
 - ▶ ...
 - ▶ The function containing v_{m-1} is called from v_m

Call String Approach — Context Interpretation

- ▶ ϵ is an empty tuple, representing the initial call context
- ▶ $(v_1, v_2, \dots, v_m)$ where $m < k$ represents call stacks of height exactly m
 - ▶ v_m must be a call node in `main`
- ▶ $(v_1, v_2, \dots, v_k)$ represents call stacks of height at least k
 - ▶ Call strings longer than k are truncated

Call String Transfer Functions — Push and Call

$$\text{push}((v_1, \dots, v_m), v_0) = \begin{cases} (v_0, v_1, \dots, v_m) & \text{if } m < k \\ (v_0, v_1, \dots, v_{m-1}) & \text{if } m = k \end{cases}$$

$x = f(e_1, \dots, e_n)$:

$$t_v(\sigma, c) = \left\{ \begin{array}{l} ([x_1 \mapsto \text{eval}(e_1, \sigma), \dots, x_n \mapsto \text{eval}(e_n, \sigma)], \text{entry}(f), c'), \\ (\sigma[x \mapsto \sigma_{\text{return}, c'}(\text{RET})], \text{after}(v), c) \end{array} \right\}$$

where

- ▶ $x_1, \dots, x_n$ are the parameters of f
- ▶ $\text{entry}(f)$ is the entry node of f
- ▶ $c' = \text{push}(c, v)$
- ▶ $\sigma_{\text{return}, c'}$ is the state at the return node of f with context c'
- ▶ $\text{after}(v)$ is the after-call node

Call String Transfer Functions — Return

return:

$$t_v(\sigma, c) = \{(\sigma_{v_1, c_1}[x_1 \mapsto \sigma(\text{RET})], \text{after}(v_1), c_1), \dots\}$$

where

- ▶ v_i is a call node and c_i is a context at v_i where $\text{push}(c_i, v_i) = c$
- ▶ σ_{v_i, c_i} is the state at v_i with c_i
- ▶ x_i is the variable assigned the return value at v_i
- ▶ $\text{after}(v_i)$ is the after-call node for v_i

Call String Example 1 ($k = 1$)

```
fn foo(a) {  
  // (C1) a: [1, 1]  
  // (C2) a: [2, 2]  
  RET = a + a;  
  // (C1) RET: [2, 2]  
  // (C2) RET: [4, 4]  
  return;  
}  
if input() {  
  x = 1;  
  // x: [1, 1]  
  y = foo(x); // C1  
  // y: [2, 2]  
} else {  
  x = 2;  
  // x: [2, 2]  
  z = foo(x); // C2  
  // z: [4, 4]  
}
```

Call String Example 2 ($k = 1$)

```
fn foo(a) {  
  // (C1) a: [1, 1]  
  
  RET = a + 1;  
  // (C1) RET: [2, 2]  
  
  return;  
}  
  
x = foo(1); // C1  
// x: [2, 2]  
y = foo(x); // C2
```

```
fn foo(a) {  
  // (C1) a: [1, 1]  
  // (C2) a: [2, 2]  
  RET = a + 1;  
  // (C1) RET: [2, 2]  
  // (C2) RET: [3, 3]  
  return;  
}  
  
x = foo(1); // C1  
// x: [2, 2]  
y = foo(x); // C2  
// y: [3, 3]
```

Call String Example 3 ($k = 1$)

```
fn bar(c) {  
  // (C3) c: [1, 2]  
  RET = c;  
  // (C3) RET: [1, 2]  
  return;  
}  
  
fn foo(a) {  
  // (C1) a: [1, 1]  
  // (C2) a: [2, 2]  
  b = bar(a); // C3  
  // (C1) b: [1, 2]  
  // (C2) b: [1, 2]  
  RET = b + b;  
  // (C1) RET: [2, 4]  
  // (C2) RET: [2, 4]  
  return;  
}
```

```
if input() {  
  x = 1;  
  // x: [1, 1]  
  y = foo(x); // C1  
  // y: [2, 4]  
} else {  
  x = 2;  
  // x: [2, 2]  
  z = foo(x); // C2  
  // z: [2, 4]  
}
```

Call String Example 4 ($k = 2$)

```
fn bar(c) {  
  // (C3, C1) c: [1, 1]  
  // (C3, C2) c: [2, 2]  
  RET = c;  
  // (C3, C1) RET: [1, 1]  
  // (C3, C2) RET: [2, 2]  
  return;  
}  
  
fn foo(a) {  
  // (C1) a: [1, 1]  
  // (C2) a: [2, 2]  
  b = bar(a); // C3  
  // (C1) b: [1, 1]  
  // (C2) b: [2, 2]  
  RET = b + b;  
  // (C1) RET: [2, 2]  
  // (C2) RET: [4, 4]  
  return;  
}
```

```
if input() {  
  x = 1;  
  // x: [1, 1]  
  y = foo(x); // C1  
  // y: [2, 2]  
} else {  
  x = 2;  
  // x: [2, 2]  
  z = foo(x); // C2  
  // z: [4, 4]  
}
```

Precision and Cost

- ▶ Larger k gives more precision, but also higher cost
- ▶ e.g., $\text{height}((\text{Call}^{\leq k} \rightarrow \text{Var} \rightarrow \text{Sign})^n) = O(c^k \cdot m \cdot n)$
 - ▶ where $c = |\text{Call}|$ and $m = |\text{Var}|$
- ▶ In practice, $k = 1$ sometimes gives inadequate precision, and $k \geq 2$ is generally too expensive
- ▶ It is common to select k individually for each call site, based on heuristics

Call String Example 5 ($k = 1$, Sign Domain)

```
fn foo(a) {  
  // (C1) a: +  
  // (C2) a: +  
  RET = a + a;  
  // (C1) RET: +  
  // (C2) RET: +  
  return;  
}
```

```
if input() {  
  x = 1;  
  // x: +  
  y = foo(x); // C1  
  // y: +  
} else {  
  x = 2;  
  // x: +  
  z = foo(x); // C2  
  // z: +  
}
```

Functional Approach — Overview

- ▶ Context = State
- ▶ A lattice element for a CFG node v is a map $m_v : \text{State} \rightarrow \text{State} \cup \{\text{unreachable}\}$
- ▶ $m_v(\sigma)$ approximates the runtime state at v , where the function containing v is called with σ as the abstract state at the call site
- ▶ If v is the exit node of a function f , m_v is a *summary* of f , mapping abstract entry states to abstract exit states
 - ▶ Transfer function: models the effect of executing a single instruction
 - ▶ Function summary: models the effect of executing an entire function

Functional Approach — Transfer Functions

$x = f(e_1, \dots, e_n)$:

$$t_v(\sigma, c) = \left\{ \begin{array}{l} (\sigma', \text{entry}(f), \sigma'), \\ (\sigma[x \mapsto \sigma_{\text{return}, \sigma'}(\text{RET})], \text{after}(v), c) \end{array} \right\}$$

where $\sigma' = [x_1 \mapsto \text{eval}(e_1, \sigma), \dots, x_n \mapsto \text{eval}(e_n, \sigma)]$

return:

$$t_v(\sigma, c) = \{(\sigma_{v_1, c_1}[x_1 \mapsto \sigma(\text{RET})], \text{after}(v_1), c_1), \dots\}$$

where

- ▶ v_i is a call node and c_i is a context at v_i where the call context is σ
- ▶ σ_{v_i, c_i} is the state at v_i with c_i
- ▶ x_i is the variable assigned the return value at v_i
- ▶ $\text{after}(v_i)$ is the after-call node for v_i

Functional Approach — Example

```
fn foo(a) {  
  // [a: +] a: +  
  RET = a + a;  
  // [a: +] RET: +  
  return;  
}
```

```
if input() {  
  x = 1;  
  // x: +  
  y = foo(x);  
  // y: +  
} else {  
  x = 2;  
  // x: +  
  z = foo(x);  
  // z: +  
}
```

Precision and Cost

- ▶ The functional approach gives optimal precision
 - ▶ As precise as if inlining all function calls
 - ▶ Completely avoids the problem with dataflow along interprocedurally invalid paths
- ▶ However, very expensive
 - ▶ e.g., $\text{State} = \text{Var} \rightarrow \text{Sign}$
 - ▶ $\text{height}((\text{State} \rightarrow \text{State} \cup \{\text{unreachable}\})^n) = O(5^m \cdot m \cdot n)$
 - ▶ where $m = |\text{Var}|$

Practical Variants

- ▶ In practice, the functional approach is often applied selectively
 - ▶ Only on some functions, or
 - ▶ Using call contexts that only consider some of the program variables
- ▶ Parameter sensitivity: call contexts are defined by the abstract values of the function parameters
 - ▶ If no pointers or global variables, it is equivalent to the current approach
- ▶ Object sensitivity: call contexts are defined by the abstract values of the receiver objects
 - ▶ Popular when analyzing object-oriented programs

Summary

- ▶ Interprocedural analysis extends intraprocedural analysis to handle function calls
- ▶ Context-insensitive analysis merges information from all call sites, leading to imprecision from interprocedurally invalid paths
- ▶ Context-sensitive analysis distinguishes different calling contexts
- ▶ The call string approach uses sequences of call sites as contexts; k controls the trade-off between precision and cost
- ▶ The functional approach uses abstract entry states as contexts, achieving optimal precision but at potentially high cost